

1. ¿Qué problema concreto resuelve un ORM? Describir al menos 3 (tres) inconvenientes que aparecen al intentar persistir objetos directamente con JDBC puro.

Un ORM resuelve el tener resuelto el pasaje de un objeto a una base de datos. En lugar de tener que programar implementaciones manuales de cómo mapear un objeto a una base de datos y tener que hacer esto de una forma independiente del motor de base de datos para evitar dolores de cabeza; se tiene un ORM que resuelve mayormente este problema y nos permite abstraernos de su implementación

Inconvenientes que aparecen al intentar persistir objetos directamente con JDBC puro:

- Migrar a un nuevo motor de bases de datos
 - Quizá si se quiere migrar de motor de base de datos y la estructura del código no está preparada para eso, haya que reprogramar muchas cosas o inclusive tener muchas dependencias que puedan causar errores
- Manejo de relaciones
 - Cargar un objeto con relaciones nos indica que hay que ver cómo manejarlas, si de una forma eager o de una forma lazy. Programar esto ya es otro dolor de cabeza
- Mayor acoplamiento
 - La lógica de persistencia queda muy mezclada con la lógica del negocio
 - 2. El modelo relacional y el modelo OO presentan tensiones conocidas como *impedance mismatch*. Identificar cómo se manifiesta cada una en el modelo dado:
 - a) Identidad: cómo es posible identificar un objeto Purchase en Java vs. en la base de datos?

En Java hay 3 formas de identificar un objeto

- Por su referencia en memoria
- Por un método equals() que define una igualdad lógica
- Por un hash

Igualmente, en el modelo dado por la cátedra, el objeto `Purchase` tiene un atributo `id`, por lo que también se puede identificar un objeto en particular utilizando ese dato

Mientras que en la base de datos, siempre se tiene una Clave Primaria que identifica inequívocamente a una fila en particular

La tensión se da en que Java tiene varias formas de identificar a una instancia de un objeto en particular, mientras que la base de datos tiene una única manera. Esto quiere decir, que si se manejara de una forma incorrecta, en Java se podrían tener dos instancias en memoria de un mismo objeto que representan a la misma compra; mientras que en la base de datos, no se repiten ids

b) Relaciones: cómo se navega de una Purchase a sus ItemService en Java vs. en SQL?

En Java se tiene una relación entre Purchase e ItemService la cual es un atributo `itemServiceList`. Por lo que en Java es tan simple como acceder a ese atributo el cual referenciará a todos los servicios de esa compra

En SQL, no existen listas como tal, por lo que para navegar de una compra a sus ItemService hay que hacer JOINS utilizando las claves foráneas

c) Herencia: ¿cómo podría representarse la jerarquía User/DriverUser/TourGuideUser en una tabla relacional?

Hay tres formas de hacerlo

1. SINGLE_TABLE

- Se mapea toda la jerarquía en una única tabla que contiene todos los atributos de todas las clases más una columna que permita discriminar el tipo de usuario. Todas las columnas exclusivas a una subclase, quedarán nulas cuando no aplique

2. JOINED

- Se crea una tabla base para User y tablas adicionales para DriverUser y TourGuideUser, las cuales tendrán una clave foránea indicando cuál es la fila en User que contiene el resto de atributos

3. TABLE_PER_CLASS

- Se genera una tabla independiente para cada clase concreta ("eliminar el padre"), duplicando las columnas de User para cada tabla de la subclase

d) Ciclos de referencia: ¿existe algún ciclo en el modelo? ¿Cómo impacta en la persistencia?

Sí, hay un ciclo entre User y Purchase, debido a que un usuario conoce todas sus compras, pero también la compra conoce a su usuario (tanto directamente como a través de las rutas de esa compra)

El impacto en la persistencia es que las operaciones en cascada podrían hacer recorridos circulares infinitos si el ORM no está pensado para manejar esas situaciones. Hay que tener en cuenta tanto borrados, cargas, sincronización y orden de guardados.

3. Describir las ventajas y desventajas concretas de usar un ORM en un proyecto como este.

Ventajas

- Resuelve la diferencia de impedancia
 - Permite mapear de forma transparente a los objetos resolviendo la gran mayoría de problemas
- Reducción de código
 - Evita escribir a mano sentencias JDBC y/o SQL para las operaciones. Reduciendo los errores y separando la persistencia de la lógica de negocio

- Independencia de la base de datos
 - Al trabajar con anotaciones y objetos en lugar de un dialecto de base de datos específico, la aplicación no queda acoplada a un motor de base de datos en particular, el ORM se encarga de generar las consultas correspondientes al motor utilizado
- Persistencia por alcance y cascadas
 - Con un ORM, se reduce el número de operaciones de guardado explícitas. Permitiendo que los guardados o borrados se puedan hacer afectando también a sus asociaciones.
- Desventajas
- Problemas de rendimiento por mala configuración
 - Por ejemplo, usar siempre cargados EAGER, el cual puede causar que se carguen una gran cantidad de datos innecesarios. También, las decisiones de resolución de las jerarquías pueden causar muchos JOINS o consultas complejas
- Pérdida de independencia
 - Para implementar el ORM via anotaciones JPA, las clases del modelo "contaminan" su código con librerías e instrucciones de persistencia
- Restricciones de diseño
 - Para que el ORM funcione, las clases deben seguir una serie de reglas estrictas

4. ¿Qué es JPA? ¿Qué nos permite definir? ¿Por qué se dice que es una especificación y no una implementación?

JPA es un estándar que define una especificación sobre cómo manejar la persistencia de objetos en Java.

Nos permite definir

- Entidades
- Claves primarias
- Relaciones
- Mapeos
- Consultas
- Comportamiento del acceso a datos y del ciclo de vida de las entidad

Se dice que es una especificación y no una implementación porque JPA no es una librería que haga el trabajo por sí sola, sino que es un conjunto de reglas e interfaces que indican cómo debe funcionar la persistencia en Java

5. ¿Cuál es la relación entre JPA e Hibernate? ¿Puede usarse JPA sin Hibernate o Hibernate sin JPA?

JPA es el estándar e Hibernate es la implementación de ese estándar

Puede usarse JPA sin hibernate, debido a que se puede elegir cualquier otra implementación o hacer una propia

También puede usarse Hibernate sin JPA, debido a que éste existía antes de JPA y se puede utilizar de forma nativa con su propia API

6. ¿Qué es la SessionFactory? ¿Qué patrón de diseño implementa? Justificar por qué se crea una sola instancia durante todo el ciclo de vida de la aplicación y no una por operación.

La SessionFactory es el componente encargado de crear objetos Session, los cuales son las sesiones utilizadas para interactuar con la base de datos

El patrón de diseño que implementa es el **Factory**, debido a que crea Session abstrayendo al cliente de saber cómo se construyen esos objetos

Además, se utiliza como Singleton porque construir una SessionFactory es costoso: se procesa la configuración y compila todos los archivos de mapeo, además es responsable de alojar y mantener estructuras globales como cachés, las cuales perderían sentido si por cada operación se creara una nueva SessionFactory

7. La SessionFactory ofrece dos formas de obtener una Session: openSession() y getCurrentSession(). Responder:

- a) ¿Cuál es la diferencia entre ambos métodos? ¿Qué ocurre con el ciclo de vida de la Session en cada caso?

openSession()

- Siempre crea una nueva Session
- Es independiente de otras sesiones
- Su ciclo de vida lo maneja el programador manualmente
- Se cierra explícitamente con close()

getCurrentSession()

- Devuelve la sesión asociada al contexto actual
- Si ya existe una en ese contexto, devuelve la misma
- Su ciclo de vida lo administra automáticamente Hibernate

- b) ¿Qué condición debe cumplirse para poder usar getCurrentSession()? ¿Que configuración requiere en Hibernate?

Para poder usar getCurrentSession() hace falta un contexto de sesión actual, normalmente ligado al thread o a la transacción

Para configurarlo, en el archivo hibernate.cfg.xml se requiere configurar la propiedad correspondiente al manejo del contexto de la sesión:

```
hibernate.current_session_context_class=thread
```

O si se usa Spring o algún framework, se puede delegar en ese framework. Utilizando Spring se configuraría

```
spring.jpa.properties.hibernate.current_session_context_class=thread
```

o directamente por integración con @Transactional

c) ¿Quién es responsable de cerrar la Session cuando se usa `getCurrentSession()`? ¿Y cuando se usa `openSession()`?

Cuando se utiliza `getCurrentSession()` la Session es cerrada automáticamente por Hibernate cuando termina la transacción o el contexto

Con `openSession()` , la Session la debe cerrar el programador manualmente llamando a `close()`

d) ¿Cuál de los dos métodos resulta más adecuado para usar en los repositorios de esta práctica y por qué?

El método más adecuado es `getCurrentSession()` , porque permite una correcta propagación de las transacciones y una mejor separación de responsabilidades. Al usarlo, la capa de servicios puede abrir una transacción y luego llamar a distintos métodos de uno o más repositorios. Todos estos repositorios compartirán la misma Session . Si se usara `openSession()` , se tendrían que abrir y cerrar sesiones constantemente, aislando operaciones y quizá necesitan pasar la sesión manualmente por parámetro a cada método, complejizando el código.

8. Completar la tabla comparativa entre JPA e Hibernate:

Aspecto	JPA	Hibernate
Tipo	Definición	Implementación
Define interfaces y anotaciones	Si	Sí
Proporciona implementación concretas	No	Sí
Genera SQL a partir de consultas JPQL/HQL	No	Sí
Maneja el Persistence Context	No	Sí

JPA

- Tipo
 - Definición/Especificación
- Define interfaces y anotaciones
 - Sí
- Proporciona implementación concreta
 - No
- Genera SQL a partir de consultas JPQL/HQL
 - No
- Maneja el Persistence Context
 - No

Hibernate

- Tipo
 - Implementación
- Define interfaces y anotaciones
 - Sí
- Proporciona implementación concreta
 - Sí
- Genera SQL a partir de consultas JPQL/HQL
 - Sí
- Maneja el Persistence Context
 - Si

9. Describir los cuatro estados posibles de una entidad en Hibernate (transient, managed, detached, removed) e indicar qué operación dispara cada transición entre ellos.

- Transient
 - El objeto acaba de ser instanciado en memoria mediante la operación `new`. No está asociado a ningún contexto de persistencia ni tiene una representación en la base de datos
- Managed
 - El objeto está asociado a una `Session` activa. Hibernate monitorea su estado y sincronizará cualquier cambio en sus atributos con la base de datos de forma automática (al momento de hacer flush o commit)
 - Un objeto pasa de Transient a Managed mediante operaciones como `save`, `persist` o por persistencia por alcance si se asocia a un objeto que ya es Managed
 - También puede pasar de Detached a Managed usando `update` o `merge`
- Detached
 - El objeto estuvo asociado a una `Session` y tiene una representación en la base de datos, pero la `Session` que lo manejaba fue cerrada o su contexto fue limpiado. Los cambios que se le hagan en memoria ya no serán sincronizados automáticamente en la base de datos
 - Un objeto pasa de Managed a Detached cuando se cierra la sesión (como con `close`), cuando finaliza la transacción al usar `getCurrentSession` o si se lo desconecta explícitamente con `evict` o `clear`
- Removed
 - El objeto ha sido marcado para ser eliminado físicamente de la base de datos al finalizar la transacción

- Un objeto pasa de Managed a Removed al invocar `delete`, `remove` o a través de operaciones en cascada de borrado

10. Describir el ciclo de vida completo de una entidad persistente, tome como ejemplo un objeto de la clase `Purchase`: desde que se instancia con `new`, pasando por su persistencia, hasta que se elimina. Indicar explícitamente en qué estado se encuentra el objeto en cada paso.

1. Instanciación

- `Purchase p = new Purchase()`
- Su estado es `Transient`
- Existe únicamente en memoria, no tiene ninguna `Session` asociada ni está en la base de datos

2. Persistencia

- `session.persist(p)`
- Su estado es `Managed`
- Hibernate empieza a administrarlo, queda asociado al contexto de persistencia
- En `flush/commit` se inserta en la db

3. Modificación

- `p.addItem(...)`
- Sigue en estado `Managed`

4. Fin de sesión

- `session.close()`
- Su estado es `Detached`
- El objeto sigue existiendo en memoria pero ya no será monitoreado para ver sus cambios
- Si cambia, no se guardarán automáticamente esos cambios

5. Reasociación

- `p = session.merge(p)`
- Su estado es `managed`
- Queda asociado a la nueva sesión

6. Eliminación

- `session.remove(p)`
- Su estado es `removed`
- Queda marcado para el borrado

7. Commit

- `transaction.commit()`
- El registro se elimina físicamente de la base de datos, destruyendo su representación persistente

Resumen

Un objeto `Purchase` comienza en estado **Transient** al crearse con `new`

Al invocar `persist` pasa a **Managed**, quedando asociado a la sesión.

Si la sesión se cierra pasa a **Detached**.

Puede volver a **Managed** mediante `merge`

Finalmente, con `remove` pasa a **Removed**, y al hacer commit se elimina físicamente de la base de datos.

11. Investigue sobre los métodos de Session: `session.save()`, `session.persist()`, `session.merge()` y `session.saveOrUpdate()`. ¿Qué permite hacer cada uno y cuál es la diferencia entre ellos? Indicar en qué estado debe estar un objeto para usar cada uno correctamente.

`session.save()`

- Guarda una entidad nueva en la base de datos
- El estado esperado del objeto es `Transient`
- Inserta un nuevo registro, genera un identificador si corresponde y pasa el objeto a `managed`
- Es un método propio de Hibernate

`session.persist()`

- Hace prácticamente lo mismo que `save()`, pero es el método estándar JPA
- El estado esperado del objeto es `Transient`
- Se marca la entidad para persistirla, luego hace un insert en un flush o commit
- El objeto pasa a `managed`
- La diferencia con `save` es
 - `persist` es estándar JPA, `save` es de Hibernate
 - `save` devuelve el id generado
 - `persist` no necesariamente ejecuta SQL inmediatamente

`session.merge()`

- Se usa para reasociar una entidad `detached`
- El estado esperado del objeto es `detached`, aunque también puede recibir `transient`
- Copia el estado del objeto recibido a una entidad `managed` y devuelve una nueva referencia `managed`
- El objeto original puede seguir `detached`, por lo que después de ejecutarlo, hay que usar la referencia devuelta

`session.saveOrUpdate()`

- Hibernate decide si hacer `INSERT` o `UPDATE`
- El estado esperado del objeto es `transient` o `detached`
- Si el objeto no existe en la db, se hace insert, si existe se hace update
- Luego de la operación queda `managed`

12. ¿Cuál es el conjunto mínimo de anotaciones que debe tener una clase para ser persistente con JPA?

Una clase cómo mínimo debe tener

1. `@Entity` : Para indicar que la clase es una entidad que debe ser mapeada a la base de datos
 2. `@Id` : Se coloca sobre un atributo para indicar que será la clave primaria del objeto
13. ¿Qué significa que JPA use persistencia por alcance (*persistence by reachability*)? ¿Qué consecuencia tiene si un objeto referenciado no está todavía persistido?

Que JPA use persistencia por alcance quiere decir que todo objeto al cual se pueda llegar a partir de un objeto que ya es persistente, entonces también debe ser persistente

Si se referencia un objeto que todavía no fue persistido, todo depende de cómo se haya configurado

1. Si se definió la propagación mediante la propiedad `cascade` , entonces el ORM detectará al objeto nuevo y lo persistirá
2. Si no se definió la propagación, entonces se arrojará una excepción

14. ¿Qué diferencia hay entre las estrategias `IDENTITY`, `SEQUENCE` y `TABLE` para la generación de IDs? ¿Cuál tiene mejor rendimiento en inserciones masivas y por qué?

IDENTITY

- La base de datos genera el ID al hacer el `INSERT`
- El ID se conoce después del `INSERT` , por lo que hibernate debe hacer la operación inmediatamente para poder conocer su identificador

SEQUENCE

- Usa una secuencia de la base de datos
- Hibernate obtiene el ID antes del insert utilizando esa secuencia
- Permite preasignar bloques de IDs, por lo que se soporta el batching
- Es la de mejor rendimiento para inserciones grandes

TABLE

- Simula la generación usando una tabla auxiliar que guarda el próximo ID
- Requiere un `select` y un `update` sobre la tabla auxiliar

- Es la opción más lenta para batching

15. Implementar el mapeo completo de la entidad Service según el diagrama. La implementación debe incluir:

- e) Clave primaria con estrategia de generación automática. Elegir entre IDENTITY, SEQUENCE o TABLE y justificar la elección.

Elegí Sequence porque es el de mejor rendimiento si se agregan varios servicios en conjunto

```
@Entity @neftalito *
@Table(name = "services")
public class Service {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
```

- f) Atributos: name (no nulo, max. 100 caracteres), description (opcional), price (no nulo).

```
@Column(nullable = false, length = 100, unique = true) 2 usages
private String name;

@Column(nullable = false) 2 usages
private float price;

@Column(nullable = true) 2 usages
private String description;
```

- g) Al menos una restricción de unicidad a nivel de columna.

```
@Column(nullable = false, length = 100, unique = true) 2 usages
private String name;
```

16. Para la relación Purchase -> ItemService (composición uno-a-muchos):

- h) ¿Qué anotaciones se necesitan en cada lado?

En la entidad Purchase se necesita la anotación @OneToMany

En la entidad ItemService se necesita la anotación @ManyToOne. Adicionalmente se incluye la anotación @JoinColumn para indicar explícitamente la columna de la clave foránea en la base de datos

- i) ¿Qué columna o tabla aparece en la base de datos para representar esta relación?

Como es una relación 1 a N, no se genera una tabla nueva, sino que aparece una columna de clave foránea en la tabla correspondiente al lado de "muchos", es decir, en ItemService

Purchase

```
@OneToMany(mappedBy = "purchase", cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)
private List<ItemService> itemServiceList;
```

ItemService

```
@ManyToOne(fetch = FetchType.LAZY) 2 usages
@JoinColumn(name = "purchase_id", nullable = false)
private Purchase purchase;
```

j) ¿Qué es mappedBy y en qué lado de la relación va? ¿Qué ocurre si se omite en ambos lados?

mappedBy es un atributo de las relaciones de JPA que se utiliza para indicar que la relación es bidireccional y definir cuál es el lado que no controla la relación. Va del lado de "uno"

Si se omite en ambos lados, JPA asume el comportamiento por defecto para este tipo de asociaciones y se generará automáticamente una tabla intermedia para gestionar la relación, generando una estructura que en situaciones de 1 a N es ineficiente.

k) ¿Es esta relación bidireccional o unidireccional según el diagrama? ¿Cómo se refleja en el código Java?

La relación es bidireccional según el diagrama

En el código de java, esto se refleja en que ambas clases se conocen y tienen referencias mutuas

Purchase tiene un atributo que es una colección de ItemService

ItemService tiene una referencia directa a la compra a la que pertenece

17. Para las relaciones Route <-> DriverUser y Route <-> TourGuideUser (muchos-a-muchos):

l) ¿Qué anotaciones se usan?

Se utiliza @ManyToMany y al ser bidireccional, la anotación se coloca en ambas entidades

En el lado que se elige como "Dueño" de la relación, se suele agregar la anotación

@JoinTable para configurar la tabla relacional resultante, mientras que en el lado débil se

utiliza el atributo mappedBy dentro de la anotación @ManyToMany para indicar que la relación ya está gestionada por la otra clase

m) ¿Qué tabla adicional genera JPA? ¿Qué columnas tiene? Definirla explícitamente usando @JoinTable.

Dado que el modelo relacional no soporta relaciones N a N de forma directa, se genera una tabla intermedia (Join Table) para cada relación. Esta tabla contiene, como mínimo, dos columnas que actúan como claves foráneas, apuntando a la clave primaria de una entidad y la clave primaria de la otra.

```
@JoinTable(
    name = "tour_guide_user_route",
    joinColumns = @JoinColumn(name = "tour_guide_user_id"),
    inverseJoinColumns = @JoinColumn(name = "route_id")
)
```

n) ¿Pueden ambas relaciones compartir la misma tabla join? ¿Por qué?

No, no deberían compartir la misma tabla, son dos relaciones de entidades conceptualmente

distintas

Una conecta rutas con choferes y la otra con guías turísticos

18. La relación Purchase -> Review es opcional (0..1). Implementar el mapeo de ambos lados.
¿Cómo se representa la opcionalidad en JPA?

Se pone el parámetro `optional = true`

```
@OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL, optional = true)
@JoinColumn(name="review_id")
private Review review;
```

19. ItemService referencia a Service (muchos-a-uno). Analizar el diagrama: ¿es navegable esta relación desde Service hacia ItemService? Justificar si conviene hacerla bidireccional o no.

Según el diagrama, la navegación entre Service e ItemService es bidireccional, por lo que sería navegable desde Service hacia ItemService

No conviene hacerla bidireccional, a un Service "no le importa" en qué items fue referenciado. Además, esto traería el problema de que al paso del tiempo, un Service puede estar en millones de ItemService, por lo que traerse un Service y querer acceder a los Items donde fue referenciado, puede tardar mucho tiempo y cargar una gran cantidad de datos.

20. Implementar el mapeo completo de las siguientes entidades con todas sus relaciones, siguiendo las anotaciones y decisiones discutidas: Supplier, Purchase, ItemService, Route, Stop y Review. Para cada relación bidireccional, incluir las anotaciones en ambos lados.

[Punto 20 de la práctica · neftalito/UNLP@14b7e4d](#)

21. ¿Para qué sirve la propiedad fetch en las anotaciones de relación? ¿Cuáles son los valores posibles? ¿Cuál es el valor por defecto para @OneToMany, para @ManyToOne y para @ManyToMany?

La propiedad fetch se utiliza para definir el tipo de carga de una relación, es decir, determinar si los objetos asociados se traen de la base de datos inmediatamente al cargar la entidad principal o si se recuperan de forma diferida sólo cuando se accede a ellos por primera vez
Los valores posibles son

- EAGER (Carga todo inmediatamente)
- LAZY (Carga diferida)

Para @OneToMany y @ManyToMany el valor por defecto es LAZY

Para @ManyToOne y @OneToOne el valor por defecto es EAGER

22. Describir ventajas y desventajas concretas de EAGER y LAZY, en términos de performance de acceso como de espacio en memoria. ¿Por qué configurar EAGER en todas las relaciones suele ser una mala idea en aplicaciones reales?

EAGER

Ventajas

- Evita múltiples consultas individuales ya que trae toda la información inmediatamente

- Puede evitar consultas adicionales si se sabe que los datos asociados van a usarse siempre

Desventajas

- Afecta severamente el rendimiento en el acceso inicial y puede ocupar mucho espacio en memoria
- Puede traer datos que no se usan

LAZY

Ventajas

- Optimiza enormemente el uso de espacio en memoria y la velocidad de acceso inicial

Desventajas

- Puede generar lo que se conoce como N+1, en donde si se itera para acceder a todos los elementos asociados, se requiere acceder a todos ellos uno por uno, generando tantas consultas como elementos haya que cargar

Configurar **EAGER** en todas las relaciones suele ser una mala idea porque se empezarán a recuperar an cascada a todos los asociados y los asociados de estos, generando así que quizá se cargue la base de datos entera en memoria

23. Para cada relación del modelo, elegir el FetchType más adecuado y justificar. Luego implemente su decisión en el proyecto tours.

Relación	Tipo de anotación	FetchType elegido	Justificación
Purchase -> User	@ManyToOne		
Purchase -> Route	@ManyToOne		
Purchase -> itemServiceList	@OneToMany		
Purchase -> Review	@OneToOne		
ItemService -> Service	@ManyToOne		
Route -> stops	@OneToMany		
Route -> drivers (DriverUser)	@ManyToMany		
Route -> tourGuides (TourGuideUser)	@ManyToMany		
User -> purchases	@OneToMany		
Service -> supplier	@ManyToOne		

En principio, haría todo lazy porque suele ser el caso general más eficiente, pero quizá pueda cambiar algunos

Purchase -> User

- LAZY
- No siempre se necesita cargar al usuario inmediatamente al cargar una compra (como por ejemplo en un listado de compras o para ver un análisis de ventas)

Purchase -> Route

- EAGER
- Tiene sentido mostrar siempre el recorrido de una compra

Purchase -> itemServiceList

- LAZY
- Al ser una colección, conviene cargarla de forma lazy

Purchase -> Review

- LAZY
- No toda compra necesariamente tiene una review y no siempre se necesita saber la review al consultar la compra

ItemService -> Service

- EAGER
- Generalmente si se obtiene un ItemService es para ver los servicios que tiene, por lo que tiene sentido traerlos siempre que se trae

Route -> Stops

- EAGER
- Tiene sentido que siempre que se vea una ruta, se quieran saber también las paradas

Route -> drivers

- LAZY
- Al ser una colección, tiene sentido que sea más eficiente ver la ruta sin necesidad de ver inmediatamente todos los conductores

Route -> tourGuides

- LAZY

- Misma razón que el anterior

User -> purchases

- LAZY
- No necesariamente siempre que se vea un usuario se van a querer ver todas sus compras

Service -> supplier

- EAGER
- Puede ser que si se quiere ver un servicio, también se quiera saber siempre quién es el que lo provee

24. ¿Cómo podría producirse una `LazyInitializationException` en el modelo? Investigue de qué representa esta excepción y escribir un escenario concreto explicando al menos formas de resolverlo sin cambiar el `FetchType` a `EAGER`.

Un `LazyInitializationException` ocurre cuando la aplicación intenta acceder a una asociación configurada con `LAZY` pero la sesión que gestionaba la entidad ya ha sido cerrada o la transacción ha terminado. Al no haber sesión activa, Hibernate no puede conectarse a la base de datos para realizar la consulta necesaria para recuperar los datos

Un ejemplo donde podría ocurrir

1. Se abre una transacción y se busca una compra por su ID. La compra tiene al usuario configurado como `LAZY`
2. El repositorio devuelve la compra y la transacción se cierra haciendo un commit, momento donde el objeto `Purchase` pasa a estado `Detached` y el contexto se destruye
3. El objeto se envía a una capa de vista, por ejemplo una página web para mostrar los detalles
4. La vista intenta acceder al Usuario que hizo la compra. Como ese dato no está cargado y no hay sesión, se lanza la excepción

Existen varias formas de resolverlo sin cambiar el fetch a `EAGER`

1. Uso de DTOs

- Se crea un objeto `PurchaseDTO`, mientras la sesión sigue abierta, se extraen los datos de la compra y de sus items y se copian al DTO. Luego, se retorna este DTO a la vista, el cual es un objeto Java simple que no fallará al ser leído fuera de la sesión

2. Uso de consultas JOIN FETCH

- Se pueden escribir consultas específicas que obliguen a traer los datos en la misma sentencia SQL. Esto anula el `LAZY` solamente para esa consulta en particular
- Ejemplo `SELECT p FROM Purchase p JOIN FETCH p.user u WHERE p.user_id = u.user_id`

3. Inicialización explícita

- Mientras la transacción y la sesión sigan activas, se puede forzar la carga invocando `Hibernate.initialize(compra.getUser())` o llamando algún método del objeto.

25. Enumerar todos los valores de `CascadeType` y explicar qué operación propaga cada uno sobre las entidades relacionadas.

- **PERSIST**
 - Si se persiste una entidad nueva, la operación se propaga y también se insertan en la base de datos las entidades asociadas que sean nuevas
- **REMOVE**
 - Si se elimina una entidad principal, también se eliminarán físicamente de la base de datos todas las entidades asociadas a ella
- **MERGE**
 - Si se vuelve a adjuntar o actualizar una entidad desconectada para sincronizar sus cambios con la base de datos, estos cambios también se fusionan y actualizan en las entidades asociadas
- **REFRESH**
 - Si se recarga el estado de una entidad desde la base de datos, esta acción de refresco se propaga para recargar también las entidades asociadas
- **DETACH**
 - Al desconectar explícitamente una entidad del contexto de persistencia, también se desconectarán de la sesión sus entidades asociadas
- **ALL**
 - Aplica todas las operaciones de cascada anteriormente descritas

26. ¿Cuál es el comportamiento por defecto cuando no se define cascade? ¿Cuál es la finalidad general de CASCADE? Proponga un caso del modelo donde definir un CASCADE inadecuado podría traer problemas a la consistencia de la base de datos.

El comportamiento por defecto es que nada se propaga. Si no se especifica el atributo `cascade` en una relación, las operaciones que se realicen sobre esa entidad, no se aplicarán automáticamente a las entidades relacionadas

La finalidad general de `cascade` es implementar el concepto de persistencia por alcance o transitividad. Determinando si una operación realizada sobre una entidad debe propagarse automáticamente a las entidades asociadas

Un ejemplo de un cascade inadecuado sería definir `REMOVE` como `cascadeType` en `ItemService` hacia `Service`, provocando que el borrado de un ítem específico de la compra de un usuario, borre también el servicio general.

27. ¿Cuál es la diferencia entre `cascade = REMOVE` y `orphanRemoval = true`? ¿Pueden usarse conjuntamente? Ejemplificar con el par `Purchase -> ItemService`.

La diferencia es que

`cascade = REMOVE`

- Actúa únicamente a nivel de la entidad padre. Significa que si la entidad principal es eliminada, el borrado se propagará y también se eliminarán de la base de datos sus entidades hijas
`orphanRemoval = true`
- Es más específico y actúa a nivel de la colección o referencia. Si una entidad hijo es removida de la colección del padre, el ORM asume que ha quedado huérfano y sin sentido de existir, entonces se borra automáticamente esa entidad huérfana

Sí, pueden usarse conjuntamente para modelar relaciones de composición estricta

Un ejemplo con `Purchase -> ItemService`

- Si sólo se usa `REMOVE`, al borrar una compra se borrarán todos sus `ItemService`. Sin embargo, si en Java se eliminara sólo un ítem de la lista, ese ítem no se borrará de la base de datos
- Si se agrega `orphanRemoval = true`, entonces al eliminar un ítem de la lista de servicios, Hibernate detectará que ese ítem fue desconectado del padre y entonces borrará ese ítem de la base de datos

28. Para la relación `Purchase -> ItemService` (composición):

o) ¿Qué tipos de `cascade` configurarías? Justificar cada uno.

Como es una composición estricta, es decir, `ItemService` no tiene sentido de que exista fuera de una compra, configuraré `CascadeType.ALL`, de forma que al guardar una compra nueva, sus ítems se inserten en la base de datos por el `PERSIST`, que al actualizarse la compra, se actualicen sus ítems con `MERGE`, y al eliminar una compra sus ítems sean destruidos `REMOVE`

p) ¿Usarías `orphanRemoval`? ¿Por qué?

Sí, usaría `orphanRemoval` porque si se quiere quitar un ítem particular de una compra, ese ítem queda desconectado de la misma, por lo que tiene sentido que sea eliminado también de la base de datos

q) Describir qué ocurre a nivel de base de datos cuando se elimina un `ItemService` de la lista de una `Purchase` y Hibernate realiza el flush.

A nivel de base de datos, al ocurrir el flush, Hibernate detecta que la entidad ha quedado huérfana. Como resultado, genera y ejecuta automáticamente una sentencia SQL `DELETE` específica apuntando al ID de ese registro en la tabla `ItemService`. Si no estuviera configurado el `orphanRemoval` el ORM pondría la clave foránea en `NULL` (lo cual fallaría si la

columna es obligatoria) o dejaría el registro como basura

29. Para la relación Purchase -> Review:

r) ¿Qué cascades configurarías?

También definiría `CascadeType.ALL`, para tener también la combinación de `PERSIST`, `MERGE`, `REMOVE`. Si un usuario realiza una reseña, esa reseña es para una compra en particular, la cual no tiene sentido que siga existiendo si la compra es eliminada. Con este `CascadeType`, las operaciones de guardado y borrado se propagan de la compra a su reseña

- s) Si se elimina una Purchase, ¿debería eliminarse también su Review? Justificar desde el modelo de negocio.

Sí, según el modelo de negocio, la reseña es opcional, pero está ligada completamente a una compra, la cual si es borrada debería también borrarse su reseña, porque por sí misma no tiene sentido

30. Para la relación Supplier -> Service:

t) ¿Qué cascades tienen sentido?

Tienen sentido `PERSIST` y `MERGE`, dado que permite que al registrar un proveedor en el sistema o al actualizar sus datos, también se inserten o actualicen automáticamente los servicios

- u) Si se elimina un Supplier, ¿qué debería ocurrir con sus Service? ¿Y con las Purchase que los contienen a través de ItemService?

Si se elimina un Supplier, sus Servicios deberían ser eliminados también, pero no utilizando un borrado en cascada con el `CascadeType.REMOVE`, dado que esto puede provocar una de dos cosas

1. El motor de base de datos arrojaría un error de violación de restricción de clave foránea
2. Si el borrado en cascada se continúa propagando, se terminarán eliminando los ItemService, destruyendo el registro histórico de las compras de los usuarios.
La solución sería hacer baja lógica en Supplier y Service para mantener la información histórica

Las compras que los contienen a través de los ItemService no deberían ser eliminadas

- 31. ¿Por que `CascadeType.REMOVE` en una relación @ManyToMany (por ejemplo Route <-> DriverUser) suele ser peligroso? Describir un escenario donde su uso cause pérdida no deseada de datos.

Es peligroso porque tanto las Rutas como los Conductores existen por sí solos. Si se tiene ese `CascadeType.REMOVE`, se van a eliminar objetos asociados que no deberían ser eliminados

Por ejemplo, si la relación Route -> drivers se configura con `CascadeType.REMOVE` y se quiere dejar de ofrecer una ruta, el borrado de esa ruta causaría también el borrado de todos

los choferes que tenían esa ruta asignada, los cuales seguramente sigan siendo empleados y no deban ser borrados.

32. Implemente las operaciones en cascada adecuada para todas las relaciones del modelo tours.

[Práctica hasta punto 32 · neftalito/UNLP@bf68f35](#)

33. Describir las tres estrategias de mapeo de herencia. Para cada una indicar qué tablas se crean, si aparece columna discriminadora y cómo resuelve Hibernate una consulta polimórfica. Completar la tabla:

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Tablas creadas en la BD			
Columna discriminadora			
NULLs en columnas de subclases			
Consulta polimórfica 'todos los Users'			
Cargar un DriverUser por ID			

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Integridad referencial (FK posibles)			
Performance en lecturas simples (de una entidad)			
Que implica para agregar nueva subclase			

- Tablas Creadas en la DB
 - SINGLE_TABLE
 - Una sola tabla para toda la jerarquía
 - JOINED
 - Una tabla por cada clase, unidas por claves foráneas
 - TABLE_PER_CLASS
 - Una tabla por clase concreta
- Columna Discriminadora
 - SINGLE_TABLE
 - Sí, es obligatorio para distinguir el tipo de instancia
 - JOINED
 - No es estrictamente necesaria
 - TABLE_PER_CLASS

- No, cada tabla representa su propio tipo
- NULLs en columnas de las subclases
 - SINGLE_TABLE
 - Sí, quedan nulas en las tuplas que no corresponden a esa subclase
 - JOINED
 - No, cada tabla contiene sólo sus propios atributos
 - TABLE_PER_CLASS
 - No, cada tabla sólo tiene los atributos correspondientes a la clase
- Consulta polimórfica "Todos los Users"
 - SINGLE_TABLE
 - Muy rápida y eficiente, sólo se lee una tabla
 - JOINED
 - Lenta y costosa, requiere ejecutar operaciones de JOIN con todas las tablas
 - TABLE_PER_CLASS
 - Muy costosa, requiere de operaciones UNION para juntar todos los resultados
- Cargar un DriverUser por ID
 - SINGLE_TABLE
 - Muy rápido, se accede directo a la tupla en la tabla única
 - JOINED
 - Regular, requiere realizar un JOIN entre la tabla User y la tabla DriversSer
 - TABLE_PER_CLASS
 - Rápido, se accede directamente a la tupla en la tabla concreta
- Integridad referencial (FK posibles)
 - SINGLE_TABLE
 - Débil, los campos de las subclases deben declararse como `nullable=true` a nivel de base de datos, impidiendo restricciones de `NOT NULL`
 - JOINED
 - Fuerte, las restricciones pueden aplicarse estrictamente en la tabla de cada subclase
 - TABLE_PER_CLASS
 - Compleja, si otras entidades apuntan por FK hacia la superclase, el id podría estar repartido en cualquier de las múltiples tablas
- Performance en lecturas simples (de una entidad)
 - SINGLE_TABLE
 - Excelente al estar todo en una misma tabla
 - JOINED
 - Menor, ya que para armar un objeto se requieren JOINS

- TABLE_PER_CLASS
 - Excelente ya que toda la información de una entidad concreta reside en una sola tabla
 - Que implica para agregar nueva subclase
 - SINGLE_TABLE
 - Obliga a alterar la estructura de la única tabla, agregando nuevas columnas y afectando al resto de clases mapeadas allí
 - JOINED
 - Crear una nueva tabla e independiente relacionada con la superclase, sin necesidad de alterar las tablas existentes
 - TABLE_PER_CLASS
 - Crear una tabla nueva completa e independiente, duplicando allí todos los atributos heredados de la superclase
34. Implementar el mapeo de la jerarquía User/DriverUser/TourGuideUser usando la estrategia SINGLE_TABLE. Especificar @Inheritance, @DiscriminatorColumn y @DiscriminatorValue para cada clase. Incluir todos los atributos.
- v) Ejecutar los tests provistos y analizar el DDL generado: ¿cuántas tablas aparecen? ¿Qué columnas tiene la tabla? ¿Dónde están los atributos expedient y education?

```
// User
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "user_type", discriminatorType =
DiscriminatorType.STRING)
public class User {
    ...
}

// DriverUser
@Entity
@DiscriminatorValue("DRIVER")
public class DriverUser extends User {
    ...
}

// TourGuideUser
@Entity
@DiscriminatorValue("TOUR_GUIDE")
public class TourGuideUser extends User {
    ...
}
```

Aparece una sola tabla

Va a contener las siguientes columnas

- id
- username
- password
- name
- email
- birthdate
- phone_number
- active
- user_type (discriminante)
- expedient
- education

Los atributos expedient y education están en la misma tabla, la diferencia es que uno de ellos va a ser null mientras el otro va a tener un valor, la forma de identificar el tipo de usuario será con el atributo `user_type`

w) ¿Qué ocurre con las columnas de subclase cuando se inserta un `TourGuideUser`? ¿Y cuando se inserta un `DriverUser`?

Al insertar un `TourGuideUser`, se guarda una sola fila en la tabla `users` con todas las columnas de la jerarquía

Lo mismo pasa cuando se inserta un `DriverUser`

La diferencia radica en qué columnas van a tener valor, qué columnas van a tener nulo y la columna discriminadora va a tener un valor u otro para identificar el tipo

x) ¿Cuáles son las ventajas y desventajas de esta estrategia para este modelo concreto?

Ventajas

- Menos tablas
- Consultas más simples y rápidas
- Polimorfismo más directo

Desventajas

- La tabla puede tener muchos NULL si las subclases tienen muchas columnas propias
- A medida que crecen las subclases, la tabla se vuelve más ancha
- Algunas restricciones de integridad son más difíciles de expresar, porque por ejemplo, hay columnas que deberían ser obligatorias sólo para una subclase en particular

- Es poco escalable si la jerarquía crece mucho
35. Reimplementar el mapeo de la misma jerarquía usando la estrategia JOINED.
- y) Ejecutar los tests y analizar el DDL: ¿cuantas tablas aparecen? ¿Qué FK existe entre ellas?

```
// User
@Entity
@Table(name="users")
@Inheritance(strategy= InheritanceType.JOINED)
public class User {
    ...
}

// DriverUser
@Entity
@Table(name = "driver_users")
public class DriverUser extends User {
    ...
}

// TourGuideUser
@Entity
@Table(name = "tour_guide_users")
public class TourGuideUser extends User {
    ...
}
```

Aparecen 3 tablas

- users
- driver_users
- tour_guide_users

Las FK que existe entre las tablas de las subclases es su id, el cual es una clave foránea al id de la tabla users

- z) Comparar el SQL generado por Hibernate al cargar un DriverUser en JOINED vs. en SINGLE_TABLE. ¿Cómo difiere?

Asumiendo que el DriverUser tiene id 58. Las diferencias son
SINGLE_TABLE

```
SELECT ...
FROM users
WHERE id = 58
```

JOINED

```
SELECT ...  
FROM users u  
JOIN driver_users d on u.id = d.id  
WHERE u.id = 58
```

aa) ¿Cuáles son las ventajas y desventajas de JOINED para este modelo?

Ventajas

- Modelo más normalizado, cada clase guarda sus atributos en su propia tabla
- No hay columnas nulas innecesarias por atributos de otras subclases
- La estructura representa mejor la jerarquía conceptual
- Suele ser mejor si la jerarquía crece o si cada subclase tiene muchos campos propios

Desventajas

- Las consultas son más complejas
 - Cargar una subclase requiere joins entre tablas
 - Insertar también implica más de una operación
 - Un fila en users
 - Una fila en la tabla hija correspondiente
 - Puede tener peor rendimiento que SINGLE_TABLE en lecturas frecuentes de subclases o consultas polimórficas
36. Realice el mismo proceso ahora con la estrategia TABLE_PER_CLASS. Indique cuál le parece la mejor estrategia para este modelo concreto y justifique su elección.

```
// User  
@Entity  
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)  
public abstract class User {  
    ...  
}  
  
// DriverUser  
@Entity  
@Table(name = "driver_users")  
public class DriverUser extends User {  
    ...  
}  
  
// TourGuideUser  
@Entity  
@Table(name = "tour_guide_users")
```



```
public class TourGuideUser extends User {  
    ...  
}
```

Aparecen dos tablas

- driver_users
- tour_guide_users

Cada tabla tiene las columnas de `user` más las columnas particulares de la subclase que representan

Ventajas

- No hay columnas nulas por atributos de otras subclases
- Cada tabla representa una clase concreta completa
- Si siempre se trabaja con subclases concretas, el diseño es mejor

Desventajas

- Duplica columnas heredadas en varias tablas
- Está menos normalizado
- Las consultas polimórficas son más costosas porque hay que hacer UNION

La mejor estrategia para mí es `JOINED`

- La jerarquía no tiene datos duplicados ni nulos innecesarios
- Cada subclase tiene pocos atributos propios
- Queda más normalizado

37. Route tiene relaciones muchos-a-muchos con DriverUser y TourGuideUser (subclases de User). Analizar el impacto de la estrategia de herencia sobre la tabla join:

bb) Si la jerarquía es `SINGLE_TABLE`, ¿a que tabla apunta la FK en la tabla join?

La FK en la tabla join apunta a la misma tabla `users`

cc) Si la jerarquía es `JOINED`, ¿cambia la tabla destino de esa FK?

Sí, cambia. La FK apunta a la tabla de cada subclase

38. ¿Qué estrategia resulta más robusta ante cambios futuros como agregar una nueva subclase de User? Justificar con al menos dos argumentos.

La estrategia más robusta es `JOINED`

La razón es que evita ensuciar una única tabla con columnas nuevas, dado que con `SINGLE_TABLE` cada vez que se agregue una nueva subclase, se van a agregar nuevas columnas a la tabla `users`, provocando muchos valores null y con un esquema menos normalizado.

Además, se mantiene mejor la normalización del modelo, dado que cada tabla tiene los

atributos específicos de cada subclase

39. Definir el patrón DAO (Data Access Object) y el patrón Repository. ¿Cuál es la diferencia conceptual más importante entre ambos? ¿En qué se diferencia su rol dentro de la arquitectura de la aplicación?

DAO

- Se utiliza típicamente para crear clases especializadas (Como `UserDAO`) enfocadas en manejar directamente los detalles técnicos de persistencia y las operaciones básicas (CRUD)
- Su responsabilidad es de ejecutar consultas y mapear resultados a objetos
- Está enfocado en la base de datos
- Su rol es más de capa de acceso a los datos y se encarga de comunicarse con la base de datos y ejecutar operaciones concretas

Repository

- Tiene como objetivo simular una colección de objetos en memoria, en la cual se entienden operaciones básicas, pero que en realidad mantiene sus elementos respaldados en una base de datos
- Su objetivo es que el acceso a datos se vea como si se estuviera trabajando con una colección de objetos del dominio, ocultando los detalles de la persistencia
- Está enfocado en el dominio
- Su rol es más de capa de dominio y se encarga de ofrecer una interfaz limpia y representar colecciones, ocultando los detalles de persistencia

40. El patrón Repository puede pensarse como una colección de objetos en memoria que además persiste su estado. Describir cómo se implementa este concepto usando Hibernate: ¿qué responsabilidades concentra un repositorio? ¿Con qué objeto de Hibernate interactúa internamente?

Con Hibernate, el repository suele implementarse como una clase o interfaz que delega las operaciones de persistencia al ORM

Ejemplo de la cátedra:

```
public class EmpleadoRepository {

    @Autowired
    private SessionFactory sessionFactory;

    public Serializable save(Object object) throws EmpleadoException {
        Session session = null;
        try {
            session = this.sessionFactory.getCurrentSession();
            return session.save(object);
        } catch (Exception e){
```

```

        if
(e.getClass().equals(org.hibernate.exception.ConstraintViolationException.class)) {
            throw new EmpleadoException("Constraint Violation");}
        else {System.out.println(e.toString());
            throw new EmpleadoException("Object can't be save");}
    }
}

public void update(Object object) {
    Session session = null;
    try {
        session = this.sessionFactory.getCurrentSession();
        session.update(object);
    } catch (Exception e){
        System.out.println(e.getMessage());
    }
}

public Optional<Empleado> getEmpleadoByLegajo(String legajo) {
    return this.sessionFactory.getCurrentSession().createQuery("from
Empleado where legajo = :legajo ").setParameter("legajo",
legajo).uniqueResultOptional();
}

public Empleado getEmpleadoById(Long s) {
    return
(Empleado)this.sessionFactory.getCurrentSession().createQuery("from Empleado
where id = :id ").setParameter("id", s).uniqueResult();
}

public HorarioLaboral getHorarioById(Integer s) {
    return (HorarioLaboral)
this.sessionFactory.getCurrentSession().createQuery("from HorarioLaboral where
id = :id").setParameter("id",s).uniqueResult();
}

public List<EmpleadoAdministrativo> getEmpleadosAdministrativo() {
    return (List<EmpleadoAdministrativo>)
this.sessionFactory.getCurrentSession().createQuery("from
EmpleadoAdministrativo").list();
}

public List getAllEmpleados() {
    return this.sessionFactory.getCurrentSession().createQuery("from

```

```

Empleado").list();
    }

    public List getEmpleadosTurnoM() {
        return this.sessionFactory.getCurrentSession().createQuery("from
Empleado e join e.horario h where h.descripcion like
:desc").setParameter("desc", "%mañana%").list();
    }
}

```

La idea es que desde el dominio parezca una colección

```

public void listarEmpleados() {
    List<Empleado> empleados = empleadoRepository.getAllEmpleados();

    for (Empleado e : empleados) {
        ...
    }
}

```

Las responsabilidades de un repository son:

- Buscar
- Guardar
- Eliminar
- Consultar de forma específica
- Abstraer el SQL

El objeto de Hibernate con el que interactúa internamente es el `Session`

41. Implementar un repositorio por cada entidad del modelo (PurchaseRepository, RouteRepository, UserRepository, ServiceRepository, SupplierRepository, ReviewRepository). Cada repositorio debe incluir al menos las operaciones básicas: guardar, buscar por ID, listar todos y eliminar. Utilizar la Session de Hibernate en cada implementación, tal como se ha explicado previamente utilizando el método `getCurrentSession()` sobre la `SessionFactory` (genere una inyección de este objeto tal como se muestra en el código de ejemplo proporcionado)

CORREGIR

Para no repetir código, hice un repositorio base tipado para que los demás repositorios puedan extender. Como se me pide que todos tengan esas operaciones básicas, pongo las operaciones básicas en el repositorio base (aunque esto aplique sólo para este caso, porque en otros casos no sería buena idea hacerlo)

```
package unlp.info.bd2.repositories;

import org.hibernate.Session;
import org.hibernate.SessionFactory;

import java.util.List;

public abstract class RepositoryBase<T> {

    protected final SessionFactory sessionFactory;

    protected RepositoryBase(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    protected Session getSession() {
        return sessionFactory.getCurrentSession();
    }

    public void save(T object) throws Exception {
        try {
            getSession().persist(object);
        } catch (Exception e) {
            if (e instanceof
org.hibernate.exception.ConstraintViolationException) {
                throw new Exception("Constraint Violation");
            } else {
                throw new Exception("Object can't be saved");
            }
        }
    }

    public T findById(Class<T> clase, Object id) {
        return getSession().get(clase, id);
    }

    public List<T> findAll(Class<T> clase) {
        return getSession()
            .createQuery("FROM " + clase.getName(), clase)
            .getResultList();
    }

    public void update(T object) {
        try {
            getSession().merge(object);
        } catch (Exception e) {
```

```

        System.out.println(e.getMessage());
    }
}

public void delete(T object) {
    try {
        getSession().remove(object);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
}

```

Así, las implementaciones concretas quedan limpias y sin código repetido

```

package unlp.info.bd2.repositories;

import org.hibernate.SessionFactory;

public class UserRepository extends RepositoryBase{
    protected UserRepository(SessionFactory sessionFactory) {
        super(sessionFactory);
    }
}

```

42. ¿Que es una transacción en el contexto de Hibernate? ¿Por qué es necesaria? ¿Qué ocurre si se realizan operaciones de escritura sin una transacción activa?

Una transacción es una unidad de trabajo **atómica**, que agrupa una o más operaciones sobre la base de datos para que actúen como bloque

Cumple las propiedades ACID

- Atomicidad: O se hacen todas las operaciones o ninguna
- Consistencia: La BD pasa de un estado válido a otro
- Aislamiento: No interfiere con otras transacciones
- Durabilidad: Una vez confirmada, persiste

En Hibernate se maneja así

```

Transaction tx = session.beginTransaction();
...
tx.commit();

```

Es necesaria porque Hibernate no ejecuta inmediatamente todo en la BD, sino que acumula cambios en el Session y recién los sincroniza (flush) en momentos clave, como en el commit.

Además, porque garantiza

- Integridad de datos
- Rollback ante errores
- Consistencia entre varias escrituras relacionadas
- Sincronización correcta del `flush`

Si se escribe sin transacción activa

- Ante un error, no hay posibilidad de rollback
- Puede haber inconsistencia de dato
- Los cambios pueden sincronizarse incorrectamente
- Errores o excepciones

43. ¿En qué capa de la aplicación debería gestionarse la transacción: en el repositorio o en la capa de servicio? Justificar la elección. ¿Qué ocurre si una misma operación necesita de varios accesos a la base de datos?

La transacción debe manejarse en la capa del servicio, el repository sólo debería encargarse de hacer las operaciones que se le solicitan, pero las transacciones deberían ser utilizadas por la capa de servicio por ser la que entiende el contexto del negocio y qué operaciones se deberían utilizar en conjunto

Si una misma operación necesita varios accesos a la base de datos:

- Si la transacción estuviera en cada repository, cada acceso haría commit por separado, y si falla uno, los anteriores quedan persistidos, rompiendo la atomicidad
- En cambio, si la transacción está en el service, varios repositories usan la misma transacción y ante un fallo hay rollback total

44. Implementar una capa de servicio que coordine las operaciones del modelo usando transacciones correctamente. Como mínimo implementar:

- a. Creación de todas las entidades persistentes.
- b. Actualización del precio de un servicio existente.
- c. Agregar un nuevo ítem a una compra existente.
- d. Eliminar una ruta existente siempre que no tenga compras asociadas.

CORREGIR

Archivos

- config/AppConfig
- Todos los repositorios
- ToursServiceImpl

45. ¿Que diferencia hay entre HQL/JPQL y SQL nativo? ¿Qué entidades, atributos y relaciones entienden HQL/JPQL que SQL no conoce directamente?

HQL/JPQL

- Trabajan sobre el modelo orientado a objetos de JPA/Hibernate
 - Entienden
 - Clases (User)
 - Atributos Java (username)
 - Relaciones mapeadas (purchase.user)
 - Herencia entre entidades
 - Colecciones de objetos
 - No hay tablas ni claves foráneas
- SQL Nativo
- Trabaja directamente sobre el modelo relacional físico
 - Entiende
 - Tablas
 - Columnas
 - Claves foráneas

Lo que entiende HQL/JPQL que SQL no conoce directamente es

- Clases
 - HQL/JPQL Entiende clases y objetos
 - SQL Sólo entiende las tablas
- Atributos Java
 - HQL/JPQL Entiende los nombres de las propiedades de cada objeto
 - SQL Sólo entiende los nombres de las columnas físicas
- Relaciones
 - HQL/JPQL Entiende cómo navegar entre asociaciones (`purchase.user.name`)
 - SQL No entiende objetos relacionados, necesita hacer JOINS
- Herencia y polimorfismo
 - HQL/JPQL Puede consultar jerarquías, por ejemplo, seleccionando desde `User` y que resuelva la jerarquía para traer también sus hijos
 - SQL No entiende subclases, sólo hay tablas

46. Implementar las siguientes consultas en los repositorios correspondientes:

CORREGIR

a. `List<Purchase> getAllPurchasesOfUsername(String username)`

Obtiene y retorna el listado de compras realizadas por el usuario con el nombre de usuario especificado por parámetro.

```
public List<Purchase> getAllPurchasesOfUsername(String username) {
    return getSession()
```



```

        .createQuery("from Purchase p where p.user.username = :username",
Purchase.class)
        .setParameter("username", username)
        .getResultList();
}

```

b. `List<User> getUserSpendingMoreThan(float mount);`

Obtiene y retorna el listado de usuarios que hayan efectuado alguna compra por un valor mayor o igual al pasado por parámetro.

```

public List<User> getUserSpendingMoreThan(float mount) {
    return getSession()
        .createQuery("""
            select p.user
            from Purchase p
            group by p.user
            having sum(p.totalPrice) >= :mount
            """, User.class)
        .setParameter("mount", mount)
        .getResultList();
}

```

c. `List<Supplier> getTopNSuppliersInPurchases(int n);`

Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de apariciones en compras.

```

public List<Supplier> getTopNSuppliersInPurchases(int n) {
    return getSession()
        .createQuery("""
            select s.supplier
            from ItemService i
            join i.service s
            group by s.supplier
            order by count(i.id) desc
            """, Supplier.class)
        .setMaxResults(n)
        .getResultList();
}

```

d. `int getCountOfPurchasesBetweenDates (Date start, Date end);`

Obtiene y retorna la cantidad de compras registradas entre dos fechas.

```

public long getCountOfPurchasesBetweenDates(Date start, Date end) {
    Long count = getSession()
        .createQuery("""
            select count(p)
            from Purchase p
            where p.date between :start and :end
        """, Long.class)
        .setParameter("start", start)
        .setParameter("end", end)
        .uniqueResult();
    return count != null ? count : 0L;
}

```

e. `List<Route> getRoutesWithStop(Stop stop);`

Obtiene y retorna el listado de rutas que incluyen una parada stop especificado por parámetro.

```

public List<Route> getRoutesWithStop(Stop stop) {
    return getSession()
        .createQuery("""
            select distinct r
            from Route r
            join r.stops s
            where s = :stop
        """, Route.class)
        .setParameter("stop", stop)
        .getResultList();
}

```

f. `int getMaxStopOfRoutes();`

Obtiene y retorna la cantidad de paradas que posee el recorrido con mayor cantidad de stops.

```

public int getMaxStopOfRoutes() {
    Integer max = getSession()
        .createQuery("""
            select max(size(r.stops))
            from Route r
        """, Integer.class)
        .uniqueResult();

    return max != null ? max : 0;
}

```

g. `List<Route> getRoutesNotSell();`

Obtiene y retorna el listado de recorridos que no fueron vendidas.

```
public List<Route> getRoutesNotSell() {  
    return getSession()  
        .createQuery("""  
            from Route r  
            where r.id not in (  
                select p.route.id  
                from Purchase p  
            )  
            """, Route.class)  
        .getResultList();  
}
```

h. `List<Route> getTop3RoutesWithMaxRating();`

Obtiene y retorna los tres recorridos con mayor promedio de rating en las reviews de compras asociadas a dicha ruta.

```
public List<Route> getTop3RoutesWithMaxRating() {  
    return getSession()  
        .createQuery("""  
            select p.route  
            from Review r  
            join r.purchase p  
            group by p.route  
            order by avg(r.rating) desc  
            """, Route.class)  
        .setMaxResults(3)  
        .getResultList();  
}
```

i. `Service getMostDemandedService();`

Obtiene y retorna el servicio que más veces fue incluido en compras, teniendo en cuenta la cantidad.

```
public Service getMostDemandedService() {  
    return getSession()  
        .createQuery("""  
            select i.service  
            from ItemService i  
            group by i.service  
        """)  
        .getResultList().get(0);  
}
```

```

        order by sum(i.quantity) desc
        """ , Service.class)
    .setMaxResults(1)
    .uniqueResult();
}

```

j. `List<TourGuideUser> getTourGuidesWithRating1();`

Obtiene y retorna el listado de guías asignados a algún tour con un rating de una estrella.

```

public List<TourGuideUser> getTourGuidesWithRating1() {
    return getSession()
        .createQuery("""
            select distinct tg
            from TourGuideUser tg
            join tg.routes r
            join Purchase p on p.route = r
            join Review rev on rev.purchase = p
            where rev.rating = 1
            """, TourGuideUser.class)
        .getResultList();
}

```

47. ¿En qué casos conviene usar una consulta SQL nativa en lugar de HQL/JPQL? Describir al menos un caso concreto del modelo donde esto sería necesario.

Conviene usar una consulta SQL nativa en lugar de HQL/JPQL cuando surge la necesidad de utilizar alguna característica o funcionalidad principal de un motor de base de datos específico, las cuales no están soportadas en JPA/Hibernate

Por ejemplo:

Si se quisiera hacer un resumen de texto de todas las paradas de una Route, separadas por coma.

Si se hiciera con HQL, el ORM traería todas las colecciones a memoria y el servidor debería armar el String concatenando una por una de las paradas a cada uno de los Route a mostrar

Con SQL nativo, se puede utilizar la función `GROUP_CONCAT` haciendo un JOIN con la tabla Stop, resolviendo la concatenación de una forma mucho más rápida